

Laboratorium 11: Pytorch Lightning

Zadanie 1: Trening i test sieci do regresji dla „toy set” z wykorzystaniem biblioteki Pytorch Lightning

Cel: Implementacja sieci do regresji z wykorzystaniem Pytorch Lightning

1. Instalacja pytorch-lightning i ray[tune]

```
pip install pytorch-lightning
pip install ray[tune]
```

2. Importy:

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import numpy as np

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset
from torch.utils.data import DataLoader

import pytorch_lightning as pl
from pytorch_lightning import Trainer

from pytorch_lightning.callbacks import ModelCheckpoint
from pytorch_lightning.callbacks import EarlyStopping
from pytorch_lightning.tuner import Tuner

from ray import tune
from ray.tune.integration.pytorch_lightning import TuneReportCallback
```

3. Implementacja dataset dla zadania regresji:

```
class MyDataset(Dataset):
    def __init__(self, data, outputs):
        """Inicjalizacja - przygotowanie struktury danych"""
        self.data = data
        self.outputs = outputs

    def __len__(self):
        """Zwraca liczbę próbek w datasetcie"""
        return len(self.data)

    def __getitem__(self, idx):
        """Zwraca próbkę o indeksie idx"""
        x = self.data[idx]
```

```
y = self.outputs[idx]
return x, y
```

4. Moduł danych pytorch-lightning:

```
class MyDataModule(pl.LightningDataModule):
    def __init__(self, batch_size=32, n_samples=5000, n_features=20, random_state=42):
        super().__init__()
        self.X, self.y = make_regression(n_samples=n_samples, n_features=n_features,
                                         random_state=random_state)
        self.X = torch.tensor(self.X, dtype=torch.float32)
        self.y = torch.tensor(self.y, dtype=torch.float32)
        self.batch_size = batch_size

    def setup(self, stage=None):
        X_train_val, X_test, y_train_val, y_test = train_test_split(self.X, self.y, test_size=0.2)
        X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.2)

        self.train_dataset = MyDataset(X_train, y_train)
        self.val_dataset = MyDataset(X_val, y_val)
        self.test_dataset = MyDataset(X_test, y_test)

    def train_dataloader(self):
        return DataLoader(self.train_dataset, batch_size=self.batch_size, shuffle=True)

    def val_dataloader(self):
        return DataLoader(self.val_dataset, batch_size=self.batch_size)

    def test_dataloader(self):
        return DataLoader(self.test_dataset, batch_size=self.batch_size)
```

5. Moduł sieci do regresji:

```
class LitModel(pl.LightningModule):
    def __init__(self, input_size, hidden_size, output_dim, lr=0.001):
        super().__init__()
        self.save_hyperparameters() # Call this first to save __init__ arguments
                                    # to self.hparams

        self.layer1 = nn.Linear(self.hparams.input_size, self.hparams.hidden_size)
        self.layer2 = nn.Linear(self.hparams.hidden_size, self.hparams.hidden_size)
        self.layer3 = nn.Linear(self.hparams.hidden_size, self.hparams.output_dim)
        self.criterion = nn.MSELoss()

    def forward(self, x):
        x = torch.relu(self.layer1(x))
        x = torch.relu(self.layer2(x))
        x = self.layer3(x)
        return x

    def training_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x).squeeze()
```

```

        loss = self.criterion(y_hat, y)
        self.log('train_loss', loss)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x).squeeze()
        loss = self.criterion(y_hat, y)
        r2 = r2_score(y.detach().cpu().numpy().squeeze(), y_hat.detach().cpu().numpy().squeeze())
        self.log('val_loss', loss, prog_bar=True)
        self.log('val_r2', r2, prog_bar=True)

    def test_step(self, batch, batch_idx):
        x, y = batch
        y_hat = self(x).squeeze()
        loss = self.criterion(y_hat, y)
        r2 = r2_score(y.detach().cpu().numpy().squeeze(), y_hat.detach().cpu().numpy().squeeze())
        self.log('test_loss', loss)
        self.log('test_r2', r2)

    def configure_optimizers(self):
        # Use the learning rate from self.hparams
        return torch.optim.Adam(self.parameters(), lr=self.hparams.lr)

```

```

model = LitModel(input_size=20, hidden_size=64, output_dim=1)

```

6. Trening i test modelu:

```

model = LitModel(input_size=20, hidden_size=64, output_dim=1)

# Trainer
trainer = Trainer(
    max_epochs=10,
    accelerator='gpu',
    devices=1
)

datamodule = MyDataModule()

trainer.fit(model, datamodule=datamodule)

trainer.test(model, datamodule=datamodule)

```

7. Callback's:

```

# Stop jeśli val_loss nie poprawia się przez 5 epochs
early_stop_callback = EarlyStopping(
    monitor='val_loss',
    patience=5,
    mode='min',
    verbose=True
)

```

```

# Zapisz best model (według val_loss)
checkpoint_callback = ModelCheckpoint(
    dirpath='./',
    filename='model-{epoch:02d}-{val_loss:.2f}',
    monitor='val_loss',
    mode='min',
    save_top_k=3, # 3 najlepsze
    save_last=True # ostatni checkpoint
)

trainer = Trainer(
    callbacks=[early_stop_callback,checkpoint_callback],
    max_epochs=100,
    accelerator='gpu',
    devices=1
)

trainer.fit(model, datamodule=datamodule)

```

8. Optymalizacja hiperparametrów

```

def train_model(config):
    model = LitModel(input_size=20,
        hidden_size=config['hidden_size'],
        output_dim=1,
        lr=config['lr']
    )

    trainer = Trainer(
        max_epochs=20,
        callbacks=[TuneReportCallback({'loss': 'val_loss'}, on='validation_end')],
    )

    trainer.fit(model, datamodule)

# Ray Tune search
analysis = tune.run(
    train_model,
    config={
        'hidden_size': tune.choice([64, 128, 256]),
        'lr': tune.loguniform(1e-4, 1e-2)
    },
    num_samples=10
)

print(analysis.get_best_config(metric='loss', mode='min'))

```

Zadanie: Zaimplementuj i wykonaj optymalizację hiperparametrów sieci do regresji dla „toy set” z wykorzystaniem pytorch-lightning.

Zadanie 2: Pytorch Lightning dla zadania regresji – dwa różne rzeczywiste zbiory danych

Cel: Implementacja i optymalizacja sieci do regresji dla rzeczywistego zbioru danych z wykorzystaniem pytorch-lightning

Studenci wykorzystują kody z Zadania 1 do implementacji i optymalizacji sieci do regresji

Zadanie: Zaimplementuj i wykonaj optymalizację hiperparametrów sieci do regresji dla rzeczywistego zbioru danych z wykorzystaniem pytorch-lightning.

Zadanie 3: Pytorch Lightning dla zadania klasyfikacji – dwa rzeczywiste zbiory danych

Cel: Implementacja i optymalizacja sieci do klasyfikacji dla rzeczywistego zbioru danych z wykorzystaniem pytorch-lightning

Studenci wykorzystują kody z Zadania 1 do implementacji i optymalizacji sieci do klasyfikacji

Zadanie: Zaimplementuj i wykonaj optymalizację hiperparametrów sieci do klasyfikacji dla rzeczywistego zbioru danych z wykorzystaniem pytorch-lightning.